OS – ORANGE PROBLEM

Simran Joshi **PES1UG24CS455** **4-H**

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227
ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
error: object is corrupt
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ find .pes/objects -ty
pe f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471
e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11
ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
```

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ xxd .pes/objects/2a/5
94d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -2
0
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  commit.c
  untracked:  pes.c
  untracked:  index.h
  untracked:  index.c
  untracked:  test_tree.c
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
 1776437187 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
 1776437187 6 file2.txt
```

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
Initialized empty PES repository in .pes/
Committed: 2a4d1116aab4... Initial commit
Committed: 2448816ce85d... Add world
Committed: 7f272c851e0c... Add farewell
commit 7f272c851e0c803befc90ad7b5a28dea10b5e9c584aa447e305f8d21d5b22181
Author: PES User <pes@localhost>
Date:   1776437482

    Add farewell

commit 2448816ce85d7acf47236332421e4799c01d775408b652817f4d501f834ae341
Author: PES User <pes@localhost>
Date:   1776437482

    Add world

commit 2a4d1116aab4dc392bbae5e0be5e5de2bd88e67520bf311008b18a0980cee52e
Author: PES User <pes@localhost>
Date:   1776437482

    Initial commit
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ find .pes -type f | s
ort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c
05e5f43
.pes/objects/24/48816ce85d7acf47236332421e4799c01d775408b652817f4d501f8
34ae341
.pes/objects/2a/4d1116aab4dc392bbae5e0be5e5de2bd88e67520bf311008b18a098
0cee52e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa80
07b5c39
.pes/objects/7f/272c851e0c803befc90ad7b5a28dea10b5e9c584aa447e305f8d21d
5b22181
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a1
0a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78
c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d1038296709
9dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a
68652bc
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
simran-joshi@simran-joshi-VirtualBox:~/OS-Orange$ cat .pes/refs/heads/m
ain
cat .pes/HEAD
7f272c851e0c803befc90ad7b5a28dea10b5e9c584aa447e305f8d21d5b22181
ref: refs/heads/main
```

Full integration test (make test-integration)

```
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: 9327b2cb717a... Initial commit

Log after first commit:
commit 9327b2cb717af9313ec4d05ee039dfcb0531b3d2d83beebb8dc683aaf5dedc42
Author: PES User <pes@localhost>
Date:   1776437616

    Initial commit


--- Second Commit ---
Committed: f71b806ef5f8... Update file.txt

--- Third Commit ---
Committed: 13b6282d95f4... Add farewell

--- Full History ---
commit 13b6282d95f4b8d615382cd7c7fba0ebdc24551b12f216e9eea4cde290cacd05
Author: PES User <pes@localhost>
Date:   1776437616
```

```
    Add farewell

commit f71b806ef5f80e7a69371ced075f5d78c3472545501beafff28f20b8175ee3ce
Author: PES User <pes@localhost>
Date:   1776437616

    Update file.txt

commit 9327b2cb717af9313ec4d05ee039dfcb0531b3d2d83beebb8dc683aaf5dedc42
Author: PES User <pes@localhost>
Date:   1776437616

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
13b6282d95f4b8d615382cd7c7fba0ebdc24551b12f216e9eea4cde290cacd05
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9
a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c
4c7cbfa
.pes/objects/13/b6282d95f4b8d615382cd7c7fba0ebdc24551b12f216e9eea4cde29
0cacd05
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797
533f50d
.pes/objects/93/27b2cb717af9313ec4d05ee039dfcb0531b3d2d83beebb8dc683aaf
5dedc42
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf95
9788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08d
e0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10
d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3
b3da6ff
.pes/objects/f7/1b806ef5f80e7a69371ced075f5d78c3472545501beafff28f20b81
75ee3ce
```

```
=== All integration tests completed ===
```

Analysis Questions

Q5.1 — Checkout Implementation

To implement pes checkout <branch>, the following changes are required:

- Update .pes/HEAD to point to the target branch (ref: refs/heads/<branch>).
- Read the commit hash from .pes/refs/heads/<branch>.
- Load the corresponding tree from the object store.
- Update the working directory:
  - Create missing files.
  - Delete extra files.
  - Replace modified files with versions from the target commit.
- Update .pes/index to match the checked-out tree.

This operation is complex because it must safely replace the entire working directory while preventing data loss from uncommitted changes.

Q5.2 — Detecting Dirty Working Directory

A dirty working directory can be detected by comparing:

- The file hash in the **index** (staged version),

- The actual file in the **working directory**.

Steps:

- For each tracked file, compute its current hash.
- Compare it with the hash stored in the index.
- If they differ, the file has uncommitted changes.

If checkout would overwrite such a file and the target branch has a different version, the operation must be refused to avoid data loss.

Q5.3 — Detached HEAD

In a detached HEAD state, HEAD points directly to a commit instead of a branch.

If commits are made:

- New commits are created but not attached to any branch.
- These commits become **unreachable** if no reference points to them.

Recovery:

- The user can create a new branch pointing to that commit hash.
- If not recovered, these commits may be removed during garbage collection.

Q6.1 — Garbage Collection Algorithm

To delete unreachable objects:

1. Start from all references (branches in .pes/refs/heads/).
2. Traverse all reachable objects:
    - Commit → parent + tree
    - Tree → child entries
3. Mark all visited objects using a **hash set**.
4. Delete all unmarked objects from .pes/objects.

For a repository with ~100,000 commits and multiple branches, GC may need to traverse **hundreds of thousands of objects**, depending on sharing between commits.

Q6.2 — GC Race Condition

Running garbage collection during a commit is dangerous due to race conditions.

Example:

- A commit writes new objects (tree/blob) but has not yet updated references.
- GC runs and does not see these objects as reachable.

- GC deletes them.
- The commit later references deleted objects → repository corruption.

Git avoids this by:

- Delaying deletion of unreachable objects,
- Using locks and safe update mechanisms,
- Ensuring objects are not removed while being referenced.